HOSPITAL MANAGEMENT SYSTEM

Sprint 1 Completion & Deliverables Report

Sprint period: 19–23 October 2026

Submission pack prepared 24 October 2026

GitHub repository

Project attribute	Value
Application	Hospital Management System (HMS)
Implementation	C++17 console application; CMake/CTest; local CSV persistence
Sprint objective	Establish and verify the initial HMS product increment, core workflows, persistence, validation and regression tests.
Team roster	Insert the official roster and contribution statement required by the course submission template.

1. Executive Summary

Sprint 1 established a working C++17 console-based Hospital Management System and expanded automated regression coverage across patient management, doctor management, appointment booking and validation, CSV persistence, menu navigation, and invalid-input handling. The repository also contains the project SRS, System Architecture Document (SAD), Software Test Plan, and a README describing scope and implementation.

This report distinguishes recorded local verification from later upstream evidence. The local terminal log after incorporating Issue #28 records 21/21 tests passing. The subsequently merged PR #90 reports the latest repository regression suite as 24/24 passing; the current main CMake configuration lists 24 CTest targets. A fresh local test log from the current main branch should be captured if the course requires terminal evidence from the submitted baseline.

2. Sprint Goal and Planned Scope

The project README specifies Sprint 1 dates as 19–23 October 2026 and describes the goal as establishing the implementation foundation and developing initial functionality against the approved SRS, architecture, test plan and assigned GitHub issues.

- Project setup and design review: C++17 structure, build configuration, core class responsibilities and repository conventions.
- Core classes and validation: Patient, Doctor, Appointment, HospitalSystem and FileManager responsibilities, required fields and identifier validation.
- CSV persistence: startup loading, missing/malformed input handling, immediate persistence, restart retention and failure handling.
- Core workflows and integration: patient and doctor operations, appointment booking/validation, menus and user-facing error handling.
- Testing and review: functional regression tests, invalid-input handling, data-integrity checks and documentation updates.

3. Delivered Product Increment

Area	Delivered increment	Repository evidence
Patient management	Registration, field validation, search, details/listing, update of permitted fields and protection against Patient ID changes.	PRs #61, #63, #67, #68, #70, #71, #85, #86

Area	Delivered increment	Repository evidence
Doctor management	Add doctor, duplicate-ID prevention, field validation, search, details/listing, update of permitted fields and Doctor ID immutability.	PRs #64, #65, #66, #72, #73, #87, #88
Appointments	Booking flow and rejection of unknown patient/doctor references; appointment date/time validation.	PRs #69, #74, #75, #76
CSV persistence	Startup loading, missing and malformed CSV handling, immediate patient-registration persistence, restart retention, and write-failure protection tests.	PRs #77, #78, #79, #80, #81, #84, #89
Console interface	Main/submenu navigation and invalid menu-input rejection/recovery.	PRs #82, #83, #90
Automated verification	CMake/CTest registration for the feature and robustness test cases, including invalid-input crash resilience.	Current main lists 24 CTest targets; PR #90 reports 24/24 passed.

4. Repository and Traceability Evidence

The repository's merged pull requests provide implementation and verification traceability. The following links are representative evidence for the major Sprint 1 increments:

- Patient registration and validation (#61, #63) — Patient registration; field validation.
- Patient search, details, and listing (#67, #68, #70, #71) — Patient search/details/listing coverage.
- Doctor registration, validation, search, and listing (#64, #65, #66, #72, #73) — Doctor registration, validation, search, details/listing.
- Appointment booking and input validation (#69, #74, #75, #76) — Booking flow, unknown patient/doctor rejection, date/time checks.
- CSV loading, malformed/missing files, persistence (#77, #78, #79, #80, #81) — Startup loading, missing/malformed file handling, immediate save and restart persistence.
- Menu navigation and invalid menu handling (#82, #83) — Main/submenu navigation and invalid options.
- State protection and patient update rules (#84, #85, #86) — Failed-operation state protection; patient detail updates and immutable patient IDs.
- Doctor update and immutable doctor IDs (#87, #88) — Doctor detail updates, validation, immutable Doctor ID.
- CSV write failure safety (#89) — Test coverage for safe handling of CSV write failures.
- Invalid-input crash resilience (#90) — Twenty invalid-input vectors; PR reports 24/24 CTest pass.

5. Test and Build Results

Evidence item	Recorded result	Interpretation
Local verification after Issue #28 integration	Build/configuration passed; TC-PAT-07 passed; TC-DOC-06 passed; full suite 21/21 passed.	Verified from the supplied terminal log before the later merged increments added three more CTest targets.
Current main CMake registration	24 CTest test names registered.	Verified by reading the current main CMakeLists.txt in GitHub.
Latest merged regression evidence	PR #90 reports 24/24 tests passed.	Upstream PR evidence; do not label it as a fresh local run unless you repeat it locally.
Whitespace and runtime data	The local log reports clean diff check and no runtime CSV changes for the CMake repair.	The user-provided output records the CMake fix commit and push.

Recommended final evidence capture: on the submitted baseline, run `cmake -S . -B build`, `cmake --build build`, and `ctest --test-dir build --output-on-failure`; save the complete terminal output, including the final 24/24 summary, if required by the instructor.

6. Project Documentation

- SRS — HMS_SRS_Final.pdf — Requirements, business rules and acceptance criteria.
- SAD — SAD.pdf — System architecture and design.

- Software Test Plan — Software_Test_Plan.pdf — Test approach and test-case traceability.
- README.md — Project overview, modules, build and sprint plan.

7. Remaining Backlog and Scope Boundaries

Sprint completion refers to the agreed Sprint 1 increment, not closure of every requirement in the product backlog. The repository still shows follow-up work in the backlog, including appointment conflict/availability and ordering requirements (for example, Issues #23–#26), plus later quality, security and usability requirements. Report these as subsequent backlog items unless the team verifies that the relevant issue has been completed and merged.

The manual-testing video is supplied as external evidence through the Google Drive link below. Other items not directly captured in this pack are a GitHub Project board screenshot, a team contribution statement in the instructor's format, and any instructor-specific cover sheet or marking-rubric form. These should be added if required by the course.

8. Conclusion

The repository evidence supports a substantial Sprint 1 increment: core patient and doctor record workflows, appointment booking/validation paths, CSV persistence and recovery tests, menu integration, and regression coverage. The supplied local verification log records 21/21 tests after Issue #28 integration; later merged main-branch evidence raises the suite to 24 tests and PR #90 reports 24/24 passing. The manual-testing video is provided through the Google Drive link below. Before final LMS submission, capture any required current-main terminal output and add the course-specific roster/project-board artifacts.

Manual Testing Video Evidence: View Sprint 1 Manual Testing Video (Google Drive)

Primary repository: https://github.com/HOSPITALMANAGEMENTSYSTEM4/Hospital-Management-System